```
Scenario: Registering online for a new Frequent Flyer account
Given Jane is not a Frequent Flyer member
When Jane registers for a new account
Then Jane should be sent a confirmation email with her account number
And Jane should receive 500 bonus points
```

Using Java and JBehave, you might write step definitions for the first two lines of this scenario like this:[4]

```
@Steps
NewMemberSteps newMember;

Persona user;

@Given("$user is not a Frequent Flyer member")
public void givenANonFrequentFlyerMember(String name) {
    user = Persona.withName(name);
    user.setStatus(NonMember);
    user.save();
}

@When("$name registers for a new account")
public void whenAUserRegistersForANewAccount(String name) {
    newMember.opensFrequentFlyerSite();
    newMember.choosesToRegister();
    newMember.entersNameAndAddress(user);
    newMember.submitsApplication();
}
```

❶ **The @Steps annotation indicates a "step library" in Thucydides.**

❷ **Details about each step are hidden inside this step library.**

The user that will be used throughout this scenario.

Create the user from a given Persona.

The user's journey through the system.

A business stakeholder or Scrum product owner should have no trouble reading and providing feedback on this. In fact, code like this is often defined and implemented collaboratively by a combination of developers and testers, UX experts, and business analysts. In some teams, the testers even take a very active role in implementing this layer, using the technical components provided by the developers.

These steps are more likely to change than the core business requirements, but they'll only need to be changed if some aspect of the application workflow changes. For example, maybe you'll need to add a step where Jane also needs to agree to a set of terms and conditions.

The Business Flow layer may contain several sublayers, depending on the complexity of the scenarios and application being tested. For example, in the `whenAUserRegistersForANewAccount()` method in the preceding code, you don't interact directly with the technical UI components. Instead, you delegate to a *step library* that describes the details of each step of the business flow. When you use Thucydides with JBehave, the `@Steps` annotation ❶ injects a step library object ❷ into the step definition class. The `choosesToRegister()` method in the `NewMemberSteps` step library, for example, could be implemented like this:

[4] This code follows the `@Given` step you saw in section 7.1.5.